Percentiles for Step Budgets and Cost, Worked Out by Hand

Why you set an agent's budget on p95/p99 and never on the mean — the whole calculation on one sample of fifteen runs.

What this document does. It takes one concrete sample of step counts from fifteen agent runs — $[2, 2, 3, 3, 3, 3, 4, 4, 4, 5, 5, 6, 8, 12, 20]$ — and computes its mean, median, p90, p95 and p99 by hand, using the linear-interpolation rule. Then it shows the one fact that matters for budgeting: the mean is dragged up by a few runaway runs, while the percentiles tell you the worst-case budget you must actually provision. Every number is reproduced by the Python program in Appendix A.

Where this sits. This is **Topic 2 of Module 3.1** (the efficiency and cost/latency pillars). Its companions: Module 0.0 Topic 1 (why the tail runs are the expensive quadratic-cost ones) and Module 3.4 (cost-tail percentiles and unit economics).

Concepts covered, in order: a sample as a distribution · the rank of a percentile · linear interpolation between ranks · mean vs median under skew · p90/p95/p99 · why the mean hides the tail · provisioning a step budget on p99.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: fifteen runs, sorted

An agent decides for itself how many steps a task takes, so "steps per run" is not a single number — it is a *distribution*. Collect it and sort it:

$$S = [\,2,\ 2,\ 3,\ 3,\ 3,\ 3,\ 4,\ 4,\ 4,\ 5,\ 5,\ 6,\ 8,\ 12,\ 20\,], \qquad n = 15.$$

Most runs are cheap (2–5 steps); a handful are not (8, 12, and one 20-step runaway). That long right tail is the entire subject of this document.

Quantity	Symbol	Value here
Sample of per-run step counts	S	15 values, sorted
Sample size	n	15
Sum of the sample	ΣS	84
Largest single run	$\max S$	20 steps
Percentile-to-rank method	—	linear interpolation

Notation. Index the sorted sample $s_0, \ldots, s_{n-1}$ (zero-based). The q-th percentile sits at the fractional rank $r = \frac{q}{100}(n-1)$; we interpolate linearly between the two neighbouring samples. (This is the default of `numpy.percentile` and most dashboards.)

Intuition

A percentile answers "what value is this fraction of runs at or below?" The median (p50) is the *typical* run; p95 is the run you are worse than only 5% of the time; p99 is your near worst case. You provision capacity, timeouts and budgets for the bad days — so you size on the high percentiles, not on the average, which no single run may ever equal.

2 Step 1 — the rank of a percentile

With $n = 15$ the largest index is $n - 1 = 14$. The fractional rank of the q-th percentile is

$$r(q) = \frac{q}{100}\,(n-1), \qquad P_q = s_{\lfloor r \rfloor} + \big(r - \lfloor r \rfloor\big)\,\big(s_{\lceil r \rceil} - s_{\lfloor r \rfloor}\big). \tag{1}$$

The first term picks the lower neighbour; the second walks the fractional distance toward the upper neighbour. When r is a whole number, the fraction is 0 and $P_q = s_r$ exactly.

Mini-example — this operation in isolation

Take p50. $r = 0.50 \cdot 14 = 7.0$, a whole number, so $P_{50} = s_7$. Counting into the sorted sample (s_0=2, …), $s_7 = 4$. The median is **4 steps**. ✓ Half the runs finish in ≤ 4 steps — a number a manager would happily quote. Hold onto it; the mean is about to disagree.

3 Step 2 — p90, p95, p99 by hand

p90. $r = 0.90 \cdot 14 = 12.6$. Lower neighbour $s_{12} = 8$, upper $s_{13} = 12$, fraction 0.6: $P_{90} = 8 + 0.6(12 - 8) = 8 + 2.4 = \mathbf{10.4}$.

p95. $r = 0.95 \cdot 14 = 13.3$. Lower $s_{13} = 12$, upper $s_{14} = 20$, fraction 0.3: $P_{95} = 12 + 0.3(20 - 12) = 12 + 2.4 = \mathbf{14.4}$.

p99. $r = 0.99 \cdot 14 = 13.86$. Lower $s_{13} = 12$, upper $s_{14} = 20$, fraction 0.86: $P_{99} = 12 + 0.86(20 - 12) = 12 + 6.88 = \mathbf{18.88}$.

Statistic	Rank r	Value	Reading
mean	—	5.600	average steps (pulled up by the tail)
median (p50)	7.00	4.000	the typical run
p90	12.60	10.400	1 run in 10 is worse
p95	13.30	14.400	1 run in 20 is worse
p99	13.86	18.880	near the worst case
max	14	20.000	the single runaway

Check the mean: $\Sigma S/n = 84/15 = 5.6$. ✓

4 Step 3 — the mean lies about the budget

Notice the gap. The **median is 4** but the **mean is 5.6** — the average is 40% larger than the typical run, and *no run at all* took exactly 5.6 steps. The cause is the right skew: one 20-step run alone contributes $20/84 = \mathbf{23.8}\%$ of all steps in the sample.

$$\boxed{\text{mean} > \text{median} \iff \text{right-skewed tail.}} \tag{2}$$

If you provisioned a step budget at the mean (6 steps, rounding up), you would kill the runs at 8, 12 and 20 steps — 3 of 15, i.e. 20% of traffic. Provisioning at p95 (≈ 15 steps) kills only the single 20-step run; at p99 (≈ 19) you keep essentially everything while still capping the truly pathological case.

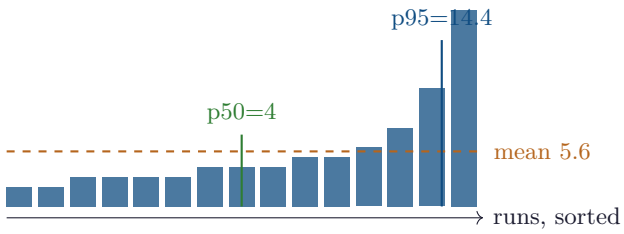
Intuition

The mean is a *accounting* number (it sums total work); it is the wrong number for *provisioning* (which is about the bad tail, not the average). The percentiles are order statistics — they ignore *how* extreme the outliers are and report *where* the ranked run sits. That robustness is exactly why SLOs are written "p99 latency < X," never "mean latency < X."

Watch out

Percentiles need enough samples to be meaningful. With $n = 15$, p99 is an interpolation between just the top two values — it is noisy, and a single new runaway moves it a lot. Report the sample size next to any p99, and prefer hundreds of runs before quoting one. The method here is correct; the *confidence* in a high percentile grows only with n.

5 The distribution, drawn



Bars are the sorted runs. The dashed mean line sits *above* most bars — pulled up by the three tall tail bars. p50 marks the typical run; p95 lives out in the tail where the budget must reach.

6 Cost cheat-sheet

Fractional rank	$r(q) = \frac{q}{100}(n-1)$
Interpolated percentile	$P_q = s_{\lfloor r \rfloor} + (r - \lfloor r \rfloor)(s_{\lceil r \rceil} - s_{\lfloor r \rfloor})$
Mean	$\bar{S} = \frac{1}{n}\sum_i s_i$
Skew signal	$\bar{S} >$ median $\Rightarrow$ right tail
Provision rule	budget $\approx$ p95 or p99, never the mean
Tail share	(top value)$/\Sigma S = 20/84 = 23.8\%$

7 An honest word about these numbers

The fifteen-run sample is illustrative and deliberately tiny so every rank is followable by hand; a real budget decision uses hundreds or thousands of runs, where p99 stops being an interpolation of two points and becomes stable. What is *not* illustrative is the lesson: agent step counts (and the token costs that ride on them) are right-skewed because a few runs loop near the budget while most exit early, so the mean systematically understates what you must provision. Size on the tail.

Appendix A — Reference implementation

```python
# percentiles.py -- reproduces every number in "Percentiles for Step Budgets".
import math

S = sorted([2,2,3,3,3,3,4,4,4,5,5,6,8,12,20])
n = len(S)

def pct(q): # linear-interpolation percentile (numpy default)
    r = q/100 * (n - 1)
    lo = math.floor(r); hi = min(lo + 1, n - 1)
    return S[lo] + (r - lo) * (S[hi] - S[lo])

mean = sum(S) / n
print(f"n={n} sum={sum(S)} mean={mean:.3f} median(p50)={pct(50):.3f}")
for q in (90, 95, 99):
    print(f"p{q} = {pct(q):.3f}")
print(f"max={max(S)} tail share of top run = {100*max(S)/sum(S):.1f}%")

# Expected output:
# n=15 sum=84 mean=5.600 median(p50)=4.000
# p90 = 10.400
# p95 = 14.400
# p99 = 18.880
# max=20 tail share of top run = 23.8%
```

— end of document —